

CHAPTER- V

OOPS BASICS

FUNCTIONS:

- A function is a block of code used to perform a specific task.
- It is also called methods in java.

SYNTAX:

```
returnType methodName(parameters) {  
    // method body (code to execute)  
    return value;  
}
```

METHODS WITHOUT PARAMETER AND RETURN VALUE:

- These methods do not take any inputs.

SYNTAX:

```
returnType methodName() {  
    //statements;  
}
```

EXAMPLE:

```
public class Example{  
    static void showYear() {  
        int year =2026;  
        System.out.println("The current year:" +year);  
    }  
}
```

```
        public static void main(String [] args){  
            showYear();  
        }  
    }  
}
```

METHOD WITH PARAMETERS AND WITHOUT RETURN VALUE:

- These methods take input values (parameters) when called.

SYNTAX:

```
returnType methodName(type para1, type para2, ...) {  
    //statements  
}
```

EXAMPLE:

```
public class Example{  
    static void addNumber(int a, int b) {  
        int sum= a+b;  
        System.out.println("The sum:"+sum);  
    }  
    public static void main(String [] args){  
        addNumber(10,20);  
    }  
}
```

METHOD WITH PARAMETER AND RETURN VALUE:

- This methods takes the input values (parameters) and it has a return value.

EXAMPLE:

```
public class Example{  
    static int sum(int a, int b) {  
        return a+b;  
    }  
    public static void main(String [] args) {  
        int result = sum(10,20);  
        System.out.println("Addition:"+result);  
    }  
}
```

METHOD WITH RETURN VALUE AND WITHOUT PARAMETER:

- This method do not any input value(parameters) and it has a return value.

EXAMPLE:

```
public class Example{  
    static int getNum() {  
        return 7;  
    }  
    Public static void main(String [] args) {  
        System.out.println("The lucky number is :"+  
getNum());  
    }  
}
```

```
    }  
}
```

TYPES OF METHODS:

- Instance method – obj.display()
- Static method- ClassName.methodName()
- Abstract method- abstract void show();
- Final method- final void display() { }
- Native method- native void methodName();

INSTANCE AND STATIC METHOD EXAMPLE:

```
class Example{  
    static void staticDisplay(){  
        System.out.println("This is static method");  
    }  
    void instanceDisplay() {  
        System.out.println("This is instance method");  
    }  
}  
  
public class MethodTypes{  
    public static void main(String [] args) {  
        staticDisplay();  
        Example e1 = new Example();  
        e1.instanceDisplay();  
    } }
```

USER-DEFINED FUNCTION:

- The user defined functions were created by user.

PRE-DEFINED FUNCTION:

- Pre defined functions were already available in the java library.

RETURN KEYWORD:

- The return keyword is used to exit a method and it send a value back to the method caller.

SYNTAX:

return value;

EXAMPLE:

```
public class Example{  
    static int sum(int a, int b) {  
        return a+b;  
    }  
    public static void main(String [] args) {  
        int result = sum(10,20);  
        System.out.println("Addition:"+result);  
    }  
}
```

COMPONENTS OF A CLASS:

- Variables(data members) – stored data
- Methods (member function) – define actions
- Constructor – initialize object (initialize variable)
- Blocks (static/ instance) – executes specific code
- Nested classes – inner class(optional)

TYPES OF VARIABLES:

- **Instance variable** – inside class, outside method(Eg: emp_name).
- **Static variable** – inside class with static keyword(Eg: static comp_name).
- **Local variable** – inside method(Eg: bonus).

EXAMPLE (INSTANCE VARIABLE):

```
class Example{
    int a=10;
    int b=20;
    void show(){
        System.out.println("a=" +a+"b="+b);
    }
}

public class ExampMain{
    public static void main(String [] args) {
        Example e1= new Example();
        e1.show();
    }
}
```

```
    }  
}
```

EXAMPLE (STATIC VARIABLE):

```
class Example{  
    static String school_name ="ABC School";  
    String name;  
}  
public classs ExampleMain{  
    public static void main(String [] args) {  
        Example e1= new Example();  
        e1.name("Abi");  
        System.out.println("The      School      name:"  
+Example.school_name+ "The name of the sudent" +e1.name);  
    }  
}
```

EXAMPLE (LOCAL VARIABLE):

```
class Examp{  
    void sum() {  
        int a= 10;  
        int b=20;  
        int sum= a+b;  
        System.out.println("The sum:" +sum);  
    }  
}
```

```
    }  
}  
  
public class ExampMain{  
    public static void main(String [] args) {  
        Examp e1 = new Examp();  
        e1.sum();  
    }  
}
```

CONSTRUCTOR:

- Constructor is a special method that is automatically called when an object of a class is created.
- Its main purpose is to initialize the object's data.

CHARACTER OF A CONSTRUCTOR:

- Same name as the class name.
- Dose not have a return type.
- It is called automatically when using the new keyword.

TYPES OF CONTSTRUCTOR:

- Default Constructor (no parameters).
- Parameterized Constructor (takes parameters).
- Copy Constructor(copies values from another object).

SYNTAX:

```
class ClassName {  
    ClassName() { //constructor  
        //initialize code  
    }  
}
```

DEFAULT CONSTRUCTOR EXAMPLE:

```
class Student {  
    String s_name;  
    int roll_no;  
    int marks;  
    Student(){  
        s_name="Abinaya";  
        roll_no= 101;  
        marks= 95;  
    }  
}  
  
public class DefaultConsEg{  
    public static void main(String [] args) {  
        Student s1= new Student();  
        System.out.println("Name:   "+s1.s_name+"   Roll   No:  
"+s1.roll_no+" Marks: "+s1.marks);  
    }  
}
```

PARAMETERIZED CONSTRUCTOR EXAMPLE:

```
class Book {  
    int book_id;  
    String b_name;  
    Book(int id, String name){  
        book_id=id;  
        b_name=name;  
    }  
}  
  
public class ParameterizedEg{  
    public static void main(String [] args) {  
        Book b1= new Book(101, "Mahabarat");  
        System.out.println(" book id: "+b1.book_id+" name of the  
book:"+b1.b_name);  
    }  
}
```

COPY CONSTRUCTOR EXAMPLE:

```
class LapTop {  
    String com_name;  
    double cost;  
    LapTop(String n, double m){  
        com_name = n;  
        cost = m;  
    }  
}
```

```

    LapTop(LapTop l){
        com_name = l.com_name;
        cost=l.cost;
    }

    void show() {
        System.out.println("Name of the Laptop:"+ com_name+" ,
Cost of the Laptop:"+cost);
    }
}

public class CopyConstEg{
    public static void main(String [] args) {
        LapTop l1= new LapTop("HP",40000);
        LapTop l2 = new LapTop(l1);
        l2.com_name="Dell";
        l1.show();
        l2.show();
    }
}

```

WAYS TO INITIALIZE OBJECT:

- Using reference variable – obj.a=10;
- Using Constructor – new Stu("Abi");
- Using method – obj.setData(10,"Abi");
- Using user input – via scanner
- Using another object – obj1=obj2;

